

TECHNICAL DOCUMENTATION

RARITONE BACKEND PROJECT

NANDINI BHIMINENI

1. INTRODUCTION

The rapid growth of e-commerce has transformed the way customers purchase fashion products. While online shopping offers convenience and accessibility, it still faces several challenges when compared to traditional in-store shopping. Customers often struggle to determine the correct size, visualize how clothing items will look on them, and receive personalized recommendations that match their preferences and body characteristics. These limitations frequently lead to customer dissatisfaction, increased product returns, and reduced confidence in online purchases.

Raritone is a virtual fashion platform developed to address these challenges by integrating personalization, digital wardrobe management, avatar support, body measurement management, and virtual try-on capabilities into a unified shopping experience. The platform aims to create a more interactive and intelligent environment where users can make informed purchasing decisions based on their individual characteristics and preferences.

The primary goal of Raritone is to bridge the gap between traditional physical shopping and online fashion retail. By enabling users to maintain personalized profiles, manage body measurements, create digital wardrobes, and access virtual fashion experiences, the platform seeks to improve customer satisfaction while reducing the uncertainty associated with online clothing purchases.

The backend system serves as the foundation of the entire platform and is responsible for handling all core business operations. It manages user authentication and authorization, product information, profile management, shopping workflows, media processing, database interactions, and communication between different platform components. The backend also provides secure and scalable RESTful APIs that allow seamless integration with web applications, mobile applications, cloud services, and future artificial intelligence modules.

The system has been developed using Node.js, Express.js, and MongoDB, which together provide a robust and efficient environment for building modern web applications. A modular architecture approach has been adopted to ensure maintainability, scalability, and ease of future enhancements. Various industry-standard practices such as JWT-based authentication, role-based access control, data validation, secure middleware implementation, and cloud-based media storage have been incorporated to improve security and performance.

The backend architecture has been specifically designed to support future innovations in fashion technology. Features such as AI-powered recommendation systems, intelligent outfit suggestions, avatar generation, virtual try-on processing, and personalized fashion analytics can be integrated into the platform with minimal modifications to the existing infrastructure. This future-ready design ensures that the platform can evolve alongside emerging technologies and changing user requirements.

The major objectives of the backend system include:

- Secure user authentication and authorization
- Product catalog management
- User profile management
- Wishlist and wardrobe management
- Shopping cart and order processing
- User measurement storage and management
- Avatar management and personalization support
- Virtual try-on request handling
- Cloud-based image and media storage
- Scalable architecture for future AI integration

Through the development of this backend system, the project demonstrates practical implementation of modern backend technologies, database management techniques, API development principles, cloud integration concepts, and software engineering best practices. The project provides a strong technological foundation for delivering an intelligent and personalized virtual fashion experience to users.

2. SYSTEM OVERVIEW

The Raritone Backend System is a centralized server-side application responsible for managing all core functionalities of the virtual fashion platform. It serves as the backbone of the system by handling business logic, database operations, authentication mechanisms, media management processes, and future artificial intelligence integration workflows. The backend ensures smooth communication between users, frontend applications, cloud services, and database resources.

The primary role of the backend is to act as an intermediary layer between users and the database. Every request made by a user is first processed by the backend, where it undergoes validation, authentication, authorization, and business logic execution before any data is stored or retrieved. This approach ensures data consistency, security, and efficient system performance.

The backend system has been designed using modern web development technologies including Node.js, Express.js, and MongoDB. It follows a modular architecture that promotes maintainability, scalability, and flexibility for future enhancements.

Major Functionalities of the Backend System

User Management

The system provides secure user management features that allow users to register, log in, manage profiles, update personal information, maintain preferences, and securely access protected resources. Authentication and authorization mechanisms ensure that only authorized users can access specific functionalities.

Product Management

The backend supports complete product management operations. Administrators can create, update, delete, and retrieve product information. Product data is organized efficiently to support searching, filtering, and future recommendation systems.

Shopping Workflow Management

The platform supports essential e-commerce functionalities including wishlist management, shopping cart operations, and order processing. These modules enable users to manage products of interest and complete purchases efficiently.

User Personalization

To provide a personalized shopping experience, the system stores user measurements, fashion preferences, wardrobe collections, and avatar-related information. This data serves as the foundation for future intelligent recommendation features.

Virtual Try-On Support

The backend includes APIs, database schemas, and workflow structures designed to support future virtual try-on capabilities. The architecture has been developed to accommodate AI-based image processing and visualization services.

Media Management

The platform supports image uploading, processing, optimization, and storage through cloud-based services. Uploaded media files are compressed and stored efficiently to improve performance and reduce storage requirements.

Security Management

Multiple security layers have been implemented to protect user data and application resources. These include JWT-based authentication, role-based access control, input validation, rate limiting, secure HTTP headers, and centralized error handling mechanisms.

Scalability Support

The backend architecture has been designed with scalability in mind. It supports future integration of technologies such as Redis caching, message queues, Docker containers, Kubernetes orchestration, microservices architecture, and AI-powered services. This ensures that the platform can handle increasing user traffic and evolving business requirements.

The backend follows RESTful API principles and uses JSON as the primary data exchange format. This standardized communication approach allows seamless integration with web applications, mobile applications, third-party services, and future artificial intelligence modules while maintaining consistency and interoperability across the platform.

3. TECHNOLOGY STACK

The Raritone Backend System was developed using modern web technologies and development tools that provide scalability, security, maintainability, and high performance. The selected technology stack enables efficient API development, database management, authentication, media processing, and future integration of advanced services such as artificial intelligence and cloud infrastructure.

Node.js

Node.js is a JavaScript runtime environment used for executing server-side applications. It is built on Google's V8 JavaScript engine and is widely used for developing scalable network applications.

Advantages

- Non-blocking and asynchronous architecture
- High performance and fast execution
- Event-driven processing model
- Efficient handling of concurrent requests
- Large ecosystem of libraries and packages
- Suitable for scalable web applications

Express.js

Express.js is a lightweight and flexible web application framework for Node.js. It simplifies the process of building RESTful APIs and backend services.

Advantages

- Lightweight and minimalistic framework
- Simplified routing and request handling

- Extensive middleware support
- Fast API development
- Easy integration with third-party libraries
- Improved code organization and maintainability

MongoDB

MongoDB is a NoSQL document-oriented database used as the primary data storage solution for the platform. It stores data in flexible JSON-like documents.

Advantages

- Flexible document-based structure
- High scalability and performance
- Easy schema modifications
- JSON-like data storage format
- Efficient handling of large datasets
- Seamless integration with Node.js applications

Mongoose

Mongoose is an Object Data Modeling (ODM) library used to interact with MongoDB. It provides schema-based data modeling and validation capabilities.

Advantages

- Schema definition and validation
- Relationship management using references
- Simplified database queries
- Data consistency and integrity
- Middleware support for database operations
- Improved code readability

JWT (JSON Web Tokens)

JWT is used to implement secure authentication and authorization mechanisms within the application.

Advantages

- Stateless authentication

- Secure API access control
- Reduced server-side session management
- Efficient user verification
- Easy implementation of role-based authorization
- Suitable for distributed applications

Google OAuth

Google OAuth is integrated to provide secure social login functionality, allowing users to authenticate using their Google accounts.

Advantages

- Simplified user registration process
- Reduced password management requirements
- Secure authentication workflow
- Improved user convenience
- Trusted identity verification

Cloudinary

Cloudinary is used as the cloud-based media management solution for storing and delivering images.

Advantages

- Secure cloud storage
- Automatic image optimization
- Content Delivery Network (CDN) support
- Reduced server storage requirements
- Scalable media management
- Fast image delivery worldwide

Multer

Multer is a middleware used for handling multipart form-data and file uploads in Express.js applications.

Advantages

- Efficient file upload handling

- Multipart form-data processing
- Easy integration with cloud storage services
- Support for image upload workflows
- Improved media management capabilities

Helmet

Helmet is a security middleware that helps protect web applications by setting various HTTP security headers.

Advantages

- Enhanced application security
- Protection against common web vulnerabilities
- Secure HTTP header configuration
- Reduced attack surface
- Improved compliance with security best practices

Express Rate Limit

Express Rate Limit is used to control the number of requests a user can make within a specified time period.

Advantages

- Protection against DDoS attacks
- Prevention of API abuse
- Better server resource management
- Enhanced application stability
- Improved overall security

Joi Validation

Joi is used for validating incoming request data before processing it within the application.

Advantages

- Input validation and sanitization
- Reduced runtime errors
- Improved data integrity
- Consistent request structure enforcement

- Enhanced application reliability

Morgan

Morgan is a middleware used for HTTP request logging and monitoring.

Advantages

- Detailed request logging
- Easier debugging and troubleshooting
- Monitoring of API activity
- Improved development visibility
- Better maintenance and performance tracking

Summary of Technology Stack

The selected technology stack provides a strong foundation for building a secure, scalable, and maintainable backend system. The combination of Node.js, Express.js, MongoDB, and supporting tools enables efficient development of RESTful APIs while ensuring high performance, security, and readiness for future enhancements such as AI integration, cloud deployment, caching systems, and microservices architecture.

4. SYSTEM ARCHITECTURE

The Raritone Backend System follows a layered and modular architecture designed to ensure maintainability, scalability, security, and ease of future enhancements. The architecture separates different responsibilities into independent layers, allowing each component to perform specific tasks without affecting other parts of the application. This design approach improves code organization, simplifies maintenance, and supports future integration of advanced technologies such as artificial intelligence and distributed systems.

The backend acts as a bridge between client applications and database services by processing requests, executing business logic, validating data, and managing secure communication.

Architecture Flow

Frontend / Mobile Application



REST API Layer



Express Application



Middleware Layer



Controller Layer



Model Layer



MongoDB Database



Cloudinary Storage



Response

Frontend Layer

The frontend layer serves as the user interaction interface and is responsible for collecting user inputs and displaying information received from the backend.

Functions

- User Registration
- User Login
- Product Browsing
- Wishlist Management
- Cart Operations
- Order Placement
- Measurement Submission
- Avatar Management
- Virtual Try-On Requests

The frontend communicates with the backend through RESTful APIs using JSON as the primary data exchange format.

API Layer

The API layer exposes endpoints that facilitate communication between frontend applications and backend services.

Responsibilities

- Receiving client requests
- Routing requests to appropriate controllers
- Processing incoming data
- Returning structured responses

The API layer follows RESTful architectural principles and ensures standardized communication across the platform.

Middleware Layer

Middleware components act as intermediate processing units between incoming requests and application logic.

Responsibilities

- JWT Authentication
- Role-Based Authorization
- Request Validation
- Error Handling
- File Upload Processing
- Security Enforcement

Benefits

- Improved security
- Better request management
- Reduced code duplication
- Centralized processing logic

Controller Layer

Controllers contain the core business logic of the application and handle operations related to different modules.

Responsibilities

- User Management

- Product Operations
- Cart Management
- Order Processing
- Measurement Handling
- Avatar Management
- Virtual Try-On Requests

Controllers communicate with database models and generate appropriate responses for client applications.

Model Layer

The model layer represents database entities and defines data structures used throughout the application.

Responsibilities

- Schema Definition
- Data Validation
- Relationship Management
- Database Operations

The project utilizes Mongoose ODM to simplify database interactions and maintain data consistency.

Database Layer

MongoDB serves as the primary database management system and stores all application data.

Responsibilities

- User Data Storage
- Product Catalog Management
- Wishlist Data
- Cart Information
- Orders
- Measurements
- Avatar Records

- Virtual Try-On Requests

Benefits

- Flexible document structure
- High scalability
- Fast query execution
- Efficient data management

Media Storage Layer

Cloudinary is used for storing and managing media files such as product images, avatars, and uploaded user content.

Responsibilities

- Image Storage
- Media Optimization
- CDN Delivery
- Asset Management

Benefits

- Faster image loading
- Reduced server storage requirements
- Global content delivery
- Improved user experience

The layered architecture enables the backend system to remain modular, scalable, and adaptable to future technological advancements.

5. PROJECT STRUCTURE

The Raritone Backend follows a modular project structure that promotes code organization, maintainability, and scalability. Each directory is dedicated to a specific responsibility within the application.

Config Directory

The config directory contains configuration files used throughout the application.

Files

- db.js

- cloudinary.js

Purpose

- MongoDB database connection
- Cloudinary configuration setup

Controllers Directory

The controllers directory contains the business logic for application modules.

Files

- authController.js
- avatarController.js
- cartController.js
- googleAuthController.js
- imageController.js
- measurementController.js
- orderController.js
- productController.js
- profileController.js
- tryOnController.js
- wardrobeController.js
- wishlistController.js

Purpose

- Process incoming requests
- Execute business logic
- Communicate with models
- Return API responses

Models Directory

The models directory contains MongoDB schema definitions.

Files

- User.js

- Profile.js
- Product.js
- Wishlist.js
- Wardrobe.js
- Cart.js
- Order.js
- Measurement.js
- Avatar.js
- TryOn.js
- ProductMapping.js
- Image.js

Purpose

- Define database structures
- Manage relationships
- Validate data
- Perform database operations

Routes Directory

The routes directory contains REST API endpoint definitions.

Purpose

- Route client requests
- Connect endpoints with controllers

Example

GET /api/products

→ productController

Middleware Directory

The middleware directory contains reusable middleware functions.

Files

- authMiddleware.js

- roleMiddleware.js
- uploadMiddleware.js
- errorMiddleware.js
- validate.js

Purpose

- Authentication
- Authorization
- Validation
- Error handling
- File upload processing

Validators Directory

The validators directory contains Joi validation schemas.

Files

- authValidator.js
- loginValidator.js
- registerValidator.js
- productValidator.js
- tryOnValidator.js
- avatarValidator.js

Purpose

- Validate incoming request data
- Ensure data integrity
- Reduce processing errors

Utils Directory

The utils directory contains reusable helper functions.

Files

- cloudinaryUpload.js
- generateToken.js

Purpose

- Cloud media operations
- Token generation
- Utility functions

Uploads Directory

The uploads directory temporarily stores uploaded files before cloud processing.

Purpose

- Temporary image storage
- Upload staging area
- File processing support

6. DATABASE DESIGN

MongoDB was selected as the primary database management system due to its flexibility, scalability, and document-oriented architecture. The database design supports multiple modules while maintaining efficient relationships between collections.

User Schema

Purpose

Stores authentication and account-related information for platform users.

Main Fields

- name
- email
- password
- role
- refreshToken

Features

- Authentication support
- Authorization support
- User identification

Relationships

User → Wishlist

User → Cart

User → Orders

User → Measurements

User → Avatar

Benefits

- Centralized user management
- Secure account handling
- Role-based access control

Product Schema

Purpose

Stores product catalog information.

Main Fields

- productName
- description
- category
- brand
- price
- image
- stock

Benefits

- Organized product management
- Product search support
- Inventory tracking

Profile Schema

Purpose

Stores user profile information and preferences.

Main Fields

- userId

- preferences
- profileImage

Benefits

- Personalized user experience
- Preference management

Wishlist Schema

Purpose

Stores products saved by users for future reference.

Main Fields

- userId
- productId

Benefits

- Product bookmarking
- Enhanced shopping experience

Wardrobe Schema

Purpose

Stores user wardrobe collections and fashion preferences.

Main Fields

- userId
- clothingItems

Benefits

- Fashion organization
- Future recommendation support

Cart Schema

Purpose

Stores products selected by users for purchase.

Main Fields

- userId

- products
- quantity

Benefits

- Shopping workflow support
- Purchase preparation

Order Schema

Purpose

Stores completed purchase information.

Main Fields

- userId
- products
- totalAmount
- orderStatus

Benefits

- Order tracking
- Purchase history management

Measurement Schema

Purpose

Stores user body measurements used for personalization.

Main Fields

- height
- weight
- chest
- waist
- hip

Benefits

- Accurate size recommendations
- Improved fitting suggestions

- Reduced product returns

Avatar Schema

Purpose

Stores avatar-related information for future virtual fashion experiences.

Main Fields

- userId
- avatarImage
- bodyType
- skinTone
- gender
- avatarModel
- status

Benefits

- Personalized avatar creation
- Virtual try-on compatibility
- AI integration readiness

TryOn Schema

Purpose

Stores virtual try-on requests and generated outputs.

Benefits

- Virtual fitting support
- AI workflow integration
- User experience enhancement

ProductMapping Schema

Purpose

Stores relationships between products and recommendation systems.

Benefits

- Product recommendations

- Compatibility analysis
- Personalized fashion suggestions
- Future AI-powered recommendation support

The database design provides a robust foundation for supporting current platform requirements while remaining flexible enough to accommodate future features and large-scale growth.

7. DATABASE RELATIONSHIPS

The Raritone Backend System utilizes MongoDB ObjectId references and Mongoose Populate functionality to establish relationships between collections. These relationships enable efficient retrieval of related data while maintaining a structured and organized database design. By using references instead of storing duplicate information, the system ensures data consistency, reduced storage requirements, and improved maintainability.

User → Wishlist

Relationship Type

One-to-Many

Purpose

A single user can save multiple products to their wishlist for future reference.

Benefits

- Personalized shopping experience
- Easy access to favorite products
- Improved customer engagement

User → Wardrobe

Relationship Type

One-to-Many

Purpose

A user can maintain multiple wardrobe items within their digital wardrobe collection.

Benefits

- Fashion organization
- Personalized styling support
- Future recommendation compatibility

User → Cart

Relationship Type

One-to-Many

Purpose

A user can add multiple products to a shopping cart before completing a purchase.

Benefits

- Purchase preparation
- Improved shopping workflow
- Convenient product management

User → Orders

Relationship Type

One-to-Many

Purpose

A user can place and maintain records of multiple orders.

Benefits

- Order history management
- Purchase tracking
- Transaction monitoring

User → Measurements

Relationship Type

One-to-One

Purpose

Stores personalized body measurements associated with a specific user.

Benefits

- Accurate size recommendations
- Personalized fitting assistance
- Reduced product returns

User → Avatar

Relationship Type

One-to-Many

Purpose

A user may create and manage multiple avatar versions for different virtual experiences.

Benefits

- Avatar customization
- Enhanced personalization
- Future AI avatar generation support

Wishlist → Product**Relationship Type**

Many-to-One

Purpose

Wishlist entries reference products stored within the product collection.

Benefits

- Efficient product retrieval
- Product information display
- Simplified wishlist management

Cart → Product**Relationship Type**

Many-to-One

Purpose

Cart items reference products selected by users for purchase.

Benefits

- Purchase processing
- Price calculation
- Inventory verification

Order → Product**Relationship Type**

Many-to-One

Purpose

Order records reference purchased products.

Benefits

- Order tracking
- Purchase history maintenance
- Transaction auditing

ProductMapping → Product

Relationship Type

Many-to-One

Purpose

Product mappings establish relationships between products for recommendation and compatibility analysis.

Benefits

- AI-powered recommendations
- Product matching
- Personalized suggestions

Mongoose Populate Functionality

The project utilizes Mongoose Populate to retrieve related documents automatically. Populate enables referenced data to be fetched in a single query, reducing development complexity and improving API usability.

Examples

- Wishlist → Product Details
- Cart → Product Details
- Orders → Product Details

Benefits

- Cleaner API responses
- Reduced database query complexity
- Improved data retrieval efficiency

- Better developer experience

8. AUTHENTICATION AND SECURITY

Security is one of the most critical aspects of the Raritone Backend System. Multiple security layers have been implemented to protect user information, application resources, APIs, and database operations. The security architecture follows modern industry practices to ensure confidentiality, integrity, and availability of data.

JWT Authentication

JSON Web Tokens (JWT) are used to implement secure user authentication.

Authentication Workflow

User Login



Credential Verification



JWT Token Generation



Token Sent to User



Access Protected APIs

Purpose

- Secure authentication
- Stateless session management
- Protected API access

Benefits

- Faster authentication process
- Improved security
- Scalability across distributed systems

Refresh Tokens

Refresh tokens are implemented to generate new access tokens without requiring users to log in repeatedly.

Workflow

Access Token Expires



Refresh Token Validation



New Access Token Generated

Benefits

- Improved user experience
- Reduced login frequency
- Enhanced session security
- Better authentication lifecycle management

Role-Based Access Control (RBAC)

The application implements role-based authorization to control access to different resources.

Supported Roles

USER

- Access personal resources
- Manage profile information
- Use platform features

ADMIN

- Product management
- Administrative operations
- Content control

SUPER_ADMIN

- Full system access
- User management
- Administrative oversight

Benefits

- Controlled resource access
- Improved security
- Administrative separation
- Reduced unauthorized operations

Google OAuth Authentication

Google OAuth integration allows users to authenticate using their Google accounts.

Workflow

User Selects Google Login



Google Authentication



User Information Retrieved



Account Creation or Login



JWT Token Generation

Benefits

- Faster registration process
- Reduced password management
- Secure authentication workflow
- Improved user convenience

Helmet Security Middleware

Helmet enhances application security by setting various HTTP security headers.

Security Features

- Content Security Policy (CSP)
- Cross-Site Scripting (XSS) Protection
- Clickjacking Protection
- MIME Type Protection

Benefits

- Reduced vulnerabilities
- Improved application security
- Enhanced protection against common attacks

Rate Limiting

Rate limiting protects APIs from abuse and excessive traffic.

Configuration

- Maximum Requests: 100
- Time Window: 15 Minutes

Benefits

- API abuse prevention
- DDoS attack mitigation
- Improved server stability
- Better resource management

Validation Layer

Joi validation schemas are used to validate incoming request data before processing.

Purpose

- Input validation
- Data verification
- Error prevention

Benefits

- Improved data quality
- Reduced invalid requests
- Enhanced application reliability

Error Handling Middleware

A centralized error handling mechanism manages all application errors.

Responsibilities

- Error capture

- Error formatting
- Error response generation
- Logging support

Benefits

- Consistent API responses
- Easier debugging
- Improved maintainability
- Better user experience

The combination of authentication, authorization, validation, security middleware, and monitoring mechanisms creates a robust security framework for the Raritone Backend System.

9. MODULE DOCUMENTATION

The backend system is organized into independent modules, each responsible for a specific business function. This modular design improves maintainability, scalability, and code reusability.

Authentication Module

Purpose

Manages user authentication and authorization.

Features

- User Registration
- User Login
- User Logout
- Forgot Password
- Reset Password
- JWT Authentication
- Refresh Tokens
- Google OAuth Integration
- Role-Based Access Control

Benefits

- Secure user access
- Protected resources
- Role management
- Enhanced authentication security

Product Module

Purpose

Manages product catalog operations.

Features

- Product Creation
- Product Updates
- Product Deletion
- Product Search
- Product Filtering

Benefits

- Efficient product management
- Inventory organization
- Simplified catalog maintenance

Profile Module

Purpose

Stores and manages user profile information.

Features

- Profile Updates
- Preference Management
- Avatar References

Benefits

- User personalization
- Better shopping experience
- Customized recommendations

Wishlist Module

Purpose

Allows users to save products for future purchases.

Features

- Add Products
- Remove Products
- View Wishlist

Benefits

- Personalized shopping
- Product bookmarking
- Increased customer engagement

Wardrobe Module

Purpose

Stores and organizes user clothing collections.

Features

- Add Clothing Items
- Remove Items
- View Wardrobe

Benefits

- Fashion organization
- Styling assistance
- Future recommendation support

Cart Module

Purpose

Provides shopping cart functionality.

Features

- Add to Cart
- Remove from Cart

- View Cart

Benefits

- Purchase preparation
- Improved shopping workflow
- Simplified checkout process

Order Module

Purpose

Manages customer order processing.

Features

- Order Creation
- Order Updates
- Order History

Benefits

- Purchase management
- Transaction tracking
- Customer order monitoring

Measurement Module

Purpose

Stores and manages user body measurements.

Features

- Save Measurements
- Retrieve Measurements
- Delete Measurements

Benefits

- Personalized fitting recommendations
- Improved size accuracy
- Reduced product returns

Avatar Module

Purpose

Stores avatar-related information for future virtual experiences.

Features

- Avatar Management
- Avatar Storage
- Avatar Status Tracking

Benefits

- Personalized avatar support
- Virtual try-on readiness
- AI integration compatibility

Try-On Module

Purpose

Supports virtual try-on workflows and future AI processing.

Features

- Try-On Requests
- User Image Handling
- Future AI Integration

Benefits

- Product visualization
- Improved purchase confidence
- Enhanced user engagement

Product Mapping Module

Purpose

Supports recommendation systems and product compatibility analysis.

Features

- Product Mapping
- Compatibility Analysis
- Recommendation Support

Benefits

- AI-powered personalization
- Smart recommendations
- Enhanced shopping experience

The modular architecture ensures that each module operates independently while working together to provide a comprehensive and scalable virtual fashion platform.

10. API DOCUMENTATION

The Raritone Backend exposes a collection of RESTful APIs that facilitate communication between frontend applications and backend services. These APIs follow standardized REST principles and use JSON as the primary format for request and response data. The APIs are designed to support authentication, product management, user personalization, shopping workflows, media management, and future AI-related functionalities.

Authentication APIs

The Authentication Module manages user registration, login, logout, password recovery, token generation, and social authentication.

Register User

Endpoint: POST /api/auth/signup

Purpose: Creates a new user account.

Request Body:

- name
- email
- password

Response:

- Success message
- User information
- JWT access token

Login User

Endpoint: POST /api/auth/login

Purpose: Authenticates users and generates access tokens.

Request Body:

- email
- password

Response:

- Access token
- Refresh token
- User information

Logout User

Endpoint: POST /api/auth/logout

Purpose: Logs out authenticated users.

Response:

- Success message

Forgot Password

Endpoint: POST /api/auth/forgot-password

Purpose: Generates password reset instructions.

Response:

- Password reset information

Reset Password

Endpoint: POST /api/auth/reset-password/:token

Purpose: Allows users to set a new password.

Response:

- Success message

Google Authentication

Endpoint: POST /api/auth/google

Purpose: Authenticates users using Google OAuth.

Response:

- User information
- Access token

Product APIs

The Product Module manages catalog operations.

Create Product

Endpoint: POST /api/products

Access: ADMIN

Purpose: Adds a new product.

Get All Products

Endpoint: GET /api/products

Purpose: Retrieves all products.

Get Product By ID

Endpoint: GET /api/products/:id

Purpose: Retrieves a specific product.

Update Product

Endpoint: PUT /api/products/:id

Access: ADMIN

Purpose: Updates product information.

Delete Product

Endpoint: DELETE /api/products/:id

Access: ADMIN

Purpose: Deletes a product.

Profile APIs

Get Profile

Endpoint: GET /api/profile

Purpose: Retrieves user profile information.

Update Profile

Endpoint: PUT /api/profile/update

Purpose: Updates profile information.

Upload Avatar

Endpoint: POST /api/profile/avatar

Purpose: Uploads profile avatar images.

Wishlist APIs

Add To Wishlist

Endpoint: POST /api/wishlist/add

Purpose: Adds products to the wishlist.

Get Wishlist

Endpoint: GET /api/wishlist

Purpose: Retrieves wishlist products.

Remove Wishlist Item

Endpoint: DELETE /api/wishlist/remove/:productId

Purpose: Removes a product from the wishlist.

Wardrobe APIs

Add Wardrobe Item

Endpoint: POST /api/wardrobe/add

Purpose: Adds clothing items to the wardrobe.

Get Wardrobe

Endpoint: GET /api/wardrobe

Purpose: Retrieves wardrobe items.

Delete Wardrobe Item

Endpoint: DELETE /api/wardrobe/:id

Purpose: Removes a wardrobe item.

Cart APIs

Add To Cart

Endpoint: POST /api/cart

Purpose: Adds products to the shopping cart.

Get Cart

Endpoint: GET /api/cart

Purpose: Retrieves cart information.

Remove Cart Item

Endpoint: DELETE /api/cart/:productId

Purpose: Removes products from the cart.

Order APIs

Create Order

Endpoint: POST /api/orders

Purpose: Creates a new order.

Get Orders

Endpoint: GET /api/orders

Purpose: Retrieves all user orders.

Get Order By ID

Endpoint: GET /api/orders/:id

Purpose: Retrieves order details.

Update Order

Endpoint: PUT /api/orders/:id

Purpose: Updates order information.

Delete Order

Endpoint: DELETE /api/orders/:id

Purpose: Deletes order records.

Measurement APIs

Save Measurements

Endpoint: POST /api/measurements

Purpose: Stores user body measurements.

Get Measurements

Endpoint: GET /api/measurements

Purpose: Retrieves measurement information.

Delete Measurements

Endpoint: DELETE /api/measurements

Purpose: Removes stored measurements.

Avatar APIs

Create Avatar

Endpoint: POST /api/avatar

Purpose: Creates avatar records.

Get Avatar

Endpoint: GET /api/avatar/:id

Purpose: Retrieves avatar information.

Get User Avatars

Endpoint: GET /api/avatar/user/:userId

Purpose: Retrieves all avatars associated with a user.

Delete Avatar

Endpoint: DELETE /api/avatar/:id

Purpose: Removes avatar information.

Try-On APIs

Create Try-On Request

Endpoint: POST /api/tryOnRoutes

Purpose: Creates virtual try-on requests.

Get Try-On Results

Endpoint: GET /api/tryOnRoutes

Purpose: Retrieves generated try-on information.

The API structure follows RESTful standards and supports secure communication between client applications and backend services.

11. MEDIA PROCESSING WORKFLOW

The Raritone Backend incorporates a structured media processing workflow to ensure efficient image handling, optimization, storage, and delivery. The workflow is designed to improve application performance while reducing storage and bandwidth requirements.

Media Processing Flow

User Uploads Image



Multer Receives File



Image Validation



Sharp Optimization



Cloudinary Upload



CDN Distribution



Response Returned

Multer

Multer is used for handling multipart form-data uploads and managing incoming image files.

Responsibilities

- File reception
- Upload validation
- Temporary storage management

Benefits

- Efficient upload handling
- Simplified file processing
- Improved upload reliability

Sharp

Sharp is used to optimize images before cloud storage.

Functions

- Image compression
- Image resizing
- Format optimization

- WebP conversion

Benefits

- Faster page loading
- Reduced storage usage
- Lower bandwidth consumption
- Improved user experience

Cloudinary

Cloudinary serves as the cloud-based media storage solution.

Functions

- Image storage
- Image transformation
- Media optimization
- CDN integration

Benefits

- High availability
- Secure cloud storage
- Scalable media management
- Fast content delivery

CDN Delivery

Cloudinary's Content Delivery Network distributes images through geographically optimized servers.

Benefits

- Reduced latency
- Faster image rendering
- Improved global accessibility
- Better application performance

The media processing pipeline ensures efficient image management while maintaining high-quality visual content across the platform.

12. TESTING DOCUMENTATION

Comprehensive testing was conducted throughout the development process to verify the reliability, functionality, security, and performance of the backend system. Testing was primarily performed using Postman, database verification tools, and application-level validation procedures.

Authentication Testing

Authentication mechanisms were tested to verify secure access control and token generation.

Verified Components

- User registration
- User login
- User logout
- Protected routes
- Google OAuth authentication

Result

Successfully Passed

CRUD Testing

All Create, Read, Update, and Delete operations were tested across major modules.

Verified Components

- Product management
- Profile management
- Wishlist operations
- Cart operations
- Order processing
- Measurement management
- Avatar management

Result

Successfully Passed

Validation Testing

Validation testing ensured that invalid requests were properly handled.

Verified Components

- Required field validation
- Invalid input handling
- Missing data validation
- Request format verification

Result

Successfully Passed

Security Testing

Security mechanisms were tested to verify proper access control and authentication.

Verified Components

- JWT authentication
- Protected API endpoints
- Role-based access control
- Token verification

Result

Successfully Passed

Database Testing

Database testing verified data integrity and relationship management.

Verified Components

- Collection relationships
- ObjectId references
- Populate functionality
- Data retrieval operations

Result

Successfully Passed

Integration Testing

Integration testing validated communication between various application layers.

Verified Components

- Route-to-controller connections
- Controller-to-model interactions
- Database communication
- API response generation

Result

Successfully Passed

Testing Summary

All major backend modules, APIs, database operations, authentication mechanisms, and security controls were tested successfully. No critical issues were identified during the testing process. The backend demonstrated stable performance, reliable functionality, and consistent API behavior across all tested modules.

13. SCALABILITY RESEARCH

As the Raritone platform continues to grow, the backend system must be capable of handling increasing numbers of users, products, transactions, media uploads, and AI-based processing requests. To ensure long-term performance and stability, several scalability strategies and technologies were researched and considered for future implementation.

Redis Caching

Redis is an in-memory data structure store used for caching frequently accessed data to improve application performance.

Potential Use Cases

- Product catalog caching
- User session caching
- Recommendation caching
- Frequently accessed API responses

Benefits

- Faster response times
- Reduced database load
- Improved system performance
- Enhanced user experience

Queue Systems

Queue systems enable asynchronous processing of time-consuming tasks, improving application responsiveness.

Potential Use Cases

- AI avatar generation
- Virtual try-on processing
- Email notifications
- Image processing
- Recommendation generation

Potential Technologies

- BullMQ
- RabbitMQ
- Apache Kafka

Benefits

- Improved scalability
- Reduced server load
- Efficient background processing
- Better resource utilization

Load Balancing

Load balancing distributes incoming traffic across multiple servers to ensure system stability and high availability.

Benefits

- High availability
- Improved performance
- Fault tolerance
- Reduced server overload

Docker

Docker enables containerization of applications for consistent deployment across different environments.

Benefits

- Environment consistency
- Simplified deployment
- Better scalability
- Easier maintenance

Kubernetes

Kubernetes is used for orchestrating containerized applications at scale.

Benefits

- Automatic scaling
- Self-healing infrastructure
- Load balancing
- Efficient resource management

MongoDB Atlas Scaling

MongoDB Atlas provides a fully managed cloud database solution with built-in scalability features.

Features

- Auto scaling
- Backup management
- Multi-region deployment
- Monitoring tools

Benefits

- High availability
- Improved reliability
- Reduced maintenance overhead
- Simplified database management

14. DEPLOYMENT DOCUMENTATION

The deployment strategy for the Raritone Backend was studied to identify suitable cloud platforms and hosting solutions for production environments.

Render

Render is a cloud platform used for deploying Node.js applications with minimal configuration.

Advantages

- Easy deployment process
- Automatic SSL configuration
- Continuous deployment support
- Developer-friendly interface

Suitability

- Small to medium-scale applications

Railway

Railway provides a modern cloud deployment platform designed for rapid development and testing.

Advantages

- Quick setup
- Simple configuration
- Integrated deployment workflow

Suitability

- Development and staging environments

AWS (Amazon Web Services)

AWS is an enterprise-grade cloud platform offering scalable infrastructure services.

Services Considered

- EC2 (Compute)
- S3 (Storage)
- Elastic Load Balancer
- CloudFront (CDN)
- EKS (Kubernetes Service)

Advantages

- Global scalability
- High reliability

- Enterprise-level infrastructure
- Extensive service ecosystem

Suitability

- Large-scale production systems

MongoDB Atlas

MongoDB Atlas is a fully managed cloud database service.

Advantages

- Automated backups
- Built-in monitoring
- Scalability features
- High availability

Cloudinary

Cloudinary is used for cloud-based media storage and optimization.

Advantages

- Image optimization
- CDN-based delivery
- Secure media storage
- Automatic transformations

15. AI INTEGRATION PLANNING

The backend architecture has been designed to support future integration of AI-powered features, enabling intelligent and personalized user experiences.

AI Avatar Generation

Workflow

User Uploads Image



Backend Receives Request



AI Processing Service



Avatar Generation



Storage in Cloudinary



Database Update



Response Returned

Benefits

- Personalized avatars
- Enhanced virtual experience
- Improved user engagement

Recommendation Engine

Workflow

User Activity Data



Measurement Data



Avatar Data



Recommendation Engine



Product Recommendations

Benefits

- Personalized shopping experience
- Increased engagement
- Improved product discovery

Virtual Try-On AI

Workflow

User Image



Avatar Data



Selected Product



AI Processing Engine



Generated Try-On Output



Cloud Storage



Response Returned

Benefits

- Better product visualization
- Reduced purchase uncertainty
- Enhanced user satisfaction

Socket.IO Real-Time System

Future real-time features can be implemented using Socket.IO for instant updates.

Potential Events

- tryon-processing
- tryon-completed
- avatar-processing
- avatar-updated
- recommendation-updated

Benefits

- Real-time user notifications

- Faster system feedback
- Improved interactivity

16. TEAM CONTRIBUTIONS

The Raritone Backend Project was developed collaboratively by a team of backend developers. Each team member contributed to specific modules while also participating in testing, documentation, integration, and overall system development activities. The collaborative approach ensured efficient development, better code quality, and a well-structured backend architecture.

Nandini Bhimineni

Role

Backend Developer & Integration Coordinator

Responsibilities

- Backend integration
- Cart module development
- Order module development
- Measurement module development
- Avatar module development
- Product mapping module development
- Database relationship design
- API testing and verification
- Cloud infrastructure research
- Scalability planning
- Documentation coordination
- AI integration planning

Vagdevi Malineni

Responsibilities

- Authentication module development
- JWT security implementation

- Role-Based Access Control (RBAC)
- Profile module development
- Wardrobe module development
- Protected routes implementation
- Security feature integration

Bharath Kumar

Responsibilities

- Authentication module support
- Wishlist module development
- API validation implementation
- Security enhancements
- Backend testing support

Vishnu Vardhan Reddy

Responsibilities

- Wishlist module development
- Virtual try-on module support
- Authentication support
- Security implementation assistance
- API testing
- Workflow validation

Meka Hrishi Teja Chowdary

Responsibilities

- Product module development
- Product API implementation
- Order module development
- Virtual try-on module support
- Product management operations
- Try-on workflow development

Nainisha Bandari

Responsibilities

- Profile module development
- Wardrobe module development
- User profile management
- User preference management
- Profile API development

Collective Team Contributions

In addition to individual responsibilities, the team collaboratively contributed to the following areas:

- REST API development
- MongoDB database design
- JWT authentication implementation
- Google OAuth integration
- Cloudinary integration for media management
- Image upload system implementation
- Error handling middleware development
- Input validation system implementation
- Security feature integration
- Technical documentation preparation
- GitHub collaboration and version control
- Postman API testing
- Backend architecture design
- AI integration planning
- Scalability research and planning
- Deployment strategy research
- Database relationship management

17. MY CONTRIBUTIONS

Contributor Information

Name: Nandini Bhimineni

Role: Backend Developer & Integration Coordinator

Overview

Throughout the Raritone Backend Project, I contributed to the development, integration, testing, documentation, research, and planning activities of the backend system. My responsibilities focused on building key backend modules, integrating completed modules into the final system, verifying database relationships, conducting testing, and preparing the platform for future AI-driven enhancements. These contributions helped ensure the system was functionally complete, well-structured, and scalable.

Modules Developed

Cart Module

Responsibilities:

- Cart API development
- Cart data management
- Product addition and removal workflows
- Cart retrieval operations

Contribution:

Developed the Cart Module to support shopping workflows, enabling users to add, view, and manage products in their cart for purchase preparation.

Order Module

Responsibilities:

- Order creation
- Order retrieval
- Order updates
- Order management workflows

Contribution:

Implemented the Order Module to handle purchase transactions, order tracking, and order history management.

Measurement Module

Responsibilities:

- Measurement storage
- Measurement retrieval
- Measurement deletion

Contribution:

Developed the Measurement Module to store user body measurements, enabling personalized recommendations and supporting future virtual try-on functionality.

Avatar Module

Responsibilities:

- Avatar schema creation
- Avatar APIs
- Avatar data management
- Documentation support

Contribution:

Developed the Avatar Module to store and manage user avatar data, enabling future AI-driven avatar generation and virtual try-on experiences.

Product Mapping Module

Responsibilities:

- Product relationship mapping
- Recommendation system preparation
- AI personalization readiness

Contribution:

Implemented Product Mapping structures to support intelligent product relationships and future recommendation engine development.

Backend Integration Activities

Responsibilities:

- Route integration
- Module integration
- Compatibility verification
- API integration testing

Contribution:

Participated in integrating individual backend modules into a unified system and ensured smooth communication between all components.

Database Design Contributions

Responsibilities:

- Relationship verification
- ObjectId reference validation
- Populate functionality testing

Contribution:

Assisted in designing and validating database relationships across modules to ensure consistency, efficiency, and proper data retrieval using Mongoose Populate.

Testing Activities

Tools Used:

- Postman

Testing Areas:

- Authentication APIs
- CRUD operations
- Protected routes
- Database operations
- Error handling

Contribution:

Performed API testing and validation to ensure all backend modules functioned correctly and met system requirements.

Documentation Activities

Responsibilities:

- Technical documentation
- Module documentation
- Architecture documentation
- README preparation

Contribution:

Prepared and maintained comprehensive project documentation to support system understanding, maintenance, and future development.

Research Activities

Cloud Infrastructure Research

Areas Studied:

- AWS
- MongoDB Atlas
- Cloudinary
- Deployment platforms

Scalability Research**Areas Studied:**

- Redis caching
- Queue systems
- Load balancing
- Docker
- Kubernetes

AI Integration Planning**Areas Studied:**

- AI avatar generation
- Recommendation systems
- Virtual try-on AI
- Socket.IO real-time systems

Contribution:

Conducted research on modern backend technologies and proposed scalable architecture improvements for future system growth and AI integration.

Key Achievements

- Contributed to multiple backend modules
- Developed and documented the Avatar Module
- Implemented Product Mapping for recommendation support
- Participated in full backend system integration
- Verified database relationships and API functionality
- Completed extensive documentation and research work
- Contributed to AI and scalability planning

Summary

My contributions focused on backend development, system integration, testing, documentation, research, and future architecture planning. These efforts helped establish a secure, scalable, modular, and AI-ready backend foundation for the Raritone platform, ensuring readiness for future enhancements and real-world deployment.

18. CONCLUSION

The Raritone Backend System successfully establishes a secure, scalable, modular, and AI-ready foundation for a modern virtual fashion platform. The system integrates key functionalities including authentication, authorization, product management, shopping workflows, user personalization, measurement handling, avatar management, virtual try-on preparation, and cloud-based media processing.

Robust security mechanisms such as JWT authentication, role-based access control, input validation, rate limiting, and middleware-based protection ensure system reliability and data safety. The modular architecture enables easy maintenance and future expansion without affecting existing functionalities.

The system has been designed with scalability as a core principle and supports future integration of advanced technologies such as Redis caching, queue systems, containerization, cloud infrastructure, AI-driven recommendation engines, and virtual try-on processing systems.

This project provided valuable hands-on experience in backend development, API design, database modeling, cloud services, security implementation, testing, and software architecture. It also strengthened practical understanding of building scalable, production-ready applications and prepared the system for future AI-powered enhancements in the fashion technology domain.